Task B1 – Implement and Instrument the File Store (7 Marks, K4)

> Run every program yourself and attach screenshots linked to the test they show. Numbers labelled **sandbox** were measured by me on a 2-core cloud machine while preparing this answer, so repeat them on your computer and report your own figures. `<fill>` marks data you must collect.

Implementation

Complete code with **TODO 1** filled in (`PassStore.java`):

```
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.StandardOpenOption;
import java.util.Arrays;

public class PassStore {
    private final Path file;

    PassStore(Path file) { this.file = file; }

    public void add(String passId, String regNo, String name, int
route, double amount)
            throws IOException {
        try (BufferedWriter w = Files.newBufferedWriter(file,
                StandardOpenOption.CREATE, StandardOpenOption.APPEND))
{
            w.write(String.join(",", passId, regNo, name,
                    String.valueOf(route), String.valueOf(amount)));
            w.newLine();
        }
    }

    /** Returns the five fields of the pass with the given ID, or null
if not found. */
    public String[] find(String passId) throws IOException {
        // TODO 1 (completed): read line by line, skip damaged lines,
return the match
        try (BufferedReader r = Files.newBufferedReader(file)) {
            String line;
            while ((line = r.readLine()) != null) {
                String[] f = line.split(",", -1);       // -1 keeps
empty trailing fields
                if (f.length != 5) continue;            // damaged
line: skip, keep searching
```

```
                if (f[0].equals(passId)) return f;
            }
        }
        return null;
    }

    public static void main(String[] args) throws IOException {
        Path f = Files.createTempFile("passes", ".csv");
        PassStore store = new PassStore(f);
        store.add("P1001", "192311001", "Arun", 12, 4500);
        Files.writeString(f, "P1002,DAMAGED LINE\n",
StandardOpenOption.APPEND);
        store.add("P1003", "192311003", "Divya", 7, 3800);
        System.out.println(Arrays.toString(store.find("P1003")));
        System.out.println(Arrays.toString(store.find("P1002")));
        Files.delete(f);
    }
}
```

Output (sandbox run, matches the expected output):

```
[P1003, 192311003, Divya, 7, 3800.0]
null
```

Why the damaged line for P1002 does not stop the search: `find` splits each line and checks `f.length != 5`; the damaged line `P1002,DAMAGED LINE` has only 2 fields, so the code executes `continue` and goes on to the next line. The loop therefore reaches P1003 and finds it. P1002 itself returns `null` because its only line is unusable.

The line counter (task A2) is in `LineCounter.java`, shown in the A2 answer.

B1.1 – Validate the implementation (`PassStoreTest.java`, sandbox output)

#	Test	Input	Actual output	What it verifies
T1	find, normal case	`find("P2")`	`[P2, R2, Divya, 7, 3800.0]`	add wrote a correct line and find reads it back
T2	find, ID not present	`find("P9")`	`null`	a missing ID returns null, with no exception
T3	find, ID of a damaged line	`find("P4")` (line `P4,only,three`)	`null`	a line with the wrong number of fields is skipped
T4	list on a route	route 12	`2 passes`	list returns

#	Test	Input	Actual output	What it verifies
				exactly the passes on that route and ignores damaged lines (including P5 whose route is x, and the empty line)
T5	find after damage	`find("P3")`	`[P3, R3, Meena, 12, 4100.0]`	damaged lines in the middle of the file do not stop the search

```
import java.io.IOException;
import java.nio.file.*;
import java.util.*;

/** Test add / find / list with a normal case, a missing ID and
damaged lines. */
public class PassStoreTest {
    static List<String[]> listOnRoute(Path file, int route) throws
IOException {
        List<String[]> out = new ArrayList<>();
        try (var r = Files.newBufferedReader(file)) {
            String line;
            while ((line = r.readLine()) != null) {
                String[] f = line.split(",", -1);
                if (f.length != 5) continue;
                try { if (Integer.parseInt(f[3]) == route) out.add(f);
}
                catch (NumberFormatException e) { /* damaged route
field: skip */ }
            }
        }
        return out;
    }
    public static void main(String[] a) throws IOException {
        Path f = Files.createTempFile("t", ".csv");
        PassStore s = new PassStore(f);
        s.add("P1", "R1", "Arun", 12, 4500);
        s.add("P2", "R2", "Divya", 7, 3800);
        s.add("P3", "R3", "Meena", 12, 4100);
        Files.writeString(f, "P4,only,three\n\nP5,R5,Kumar,x,100\n",
```

```
StandardOpenOption.APPEND);
        System.out.println("T1 find existing P2      : " +
Arrays.toString(s.find("P2")));
        System.out.println("T2 find missing P9       : " +
Arrays.toString(s.find("P9")));
        System.out.println("T3 find damaged P4       : " +
Arrays.toString(s.find("P4")));
        System.out.println("T4 list route 12         : " +
listOnRoute(f, 12).size() + " passes");
        System.out.println("T5 find P3 after damage  : " +
Arrays.toString(s.find("P3")));
        Files.delete(f);
    }
}
```

B1.2 – Compare with the Part A design

Part A design (A1)	Implementation	Change and reason
7 fields (stop and validity date included)	5 fields: `passId, regNo, name, route, amount` (as given in the starter code)	the given program uses the five-field subset; the other two fields would be appended the same way
Quote-aware splitting so a name may contain a comma	plain `line.split(",", -1)`	the starter code cannot handle `"Divya, S."`; a comma in a name would make the line look damaged (6 fields) and be skipped. Fix: escape or replace the delimiter on input, or use a quote-aware parser
`FileWriter` + `BufferedWriter`	`Files.newBufferedWriter(file, CREATE, APPEND)`	the NIO call combines both, and uses UTF-8
`-1` limit not needed in pseudocode	`split(",", -1)`	without `-1`, `split` removes trailing empty strings, so a line ending with an empty field would show too few fields
validate numbers when reading	`find` checks only the field count	a line with a non-numeric amount still counts as valid for `find`; the number checks are done in `PassValidator` (Task

Part A design (A1)	Implementation	Change and reason
		B2)

B1.3 – Measure the effect of buffering

`BufferBench.java` builds files of 1, 10 and 50 MB (lines of 37 bytes) and counts lines three ways. **Sandbox results** (repeat on your machine and report your own):

File size	Unbuffered `read()`	`byte[8192]`	`BufferedInputStream`	Lines counted
1 MB	258 ms	1 ms	24 ms	29,128
10 MB	2,459 ms	4 ms	200 ms	291,272
50 MB	14,220 ms	22 ms	886 ms	1,456,356

All three methods gave the same line count in every case (checked in the program).

Comparison with the prediction in A2.1: I predicted about 52.4 million calls for 50 MB with a cost of roughly 0.3 µs per call, i.e. about 15 s. The measured 14.2 s on my machine (= 0.27 µs per byte) agrees well, and time grew linearly with size (≈ 10× for 10× the size, ≈ 5.8× for 5× the size). Buffering with an 8 KB array was about **650 times faster** at 50 MB (14,220 ms vs 22 ms), consistent with 8,192 times fewer system calls and the remaining cost being a plain loop over memory. `BufferedInputStream` removes the system calls but still makes 52.4 million (synchronised) method calls, so it is slower than the `byte[]` loop but still ≈ 16 times faster than unbuffered. `<fill>` your own values.

```
import java.io.*;
import java.nio.file.*;

public class BufferBench {
    static Path make(int mb) throws IOException {
        Path p = Files.createTempFile("log" + mb, ".txt");
        String line = "192311001,2026-11-12,PRESENT,Hall-3\n";
// 37 bytes
        long target = mb * 1024L * 1024L;
        try (BufferedWriter w = Files.newBufferedWriter(p)) {
            for (long written = 0; written < target; written +=
line.length()) w.write(line);
        }
        return p;
    }
    public static void main(String[] args) throws IOException {
        for (int mb : new int[]{1, 10, 50}) {
            Path p = make(mb);
            String f = p.toString();
            LineCounter.countBuffered(f);
// warm-up
            long t0 = System.nanoTime(); long a =
LineCounter.countUnbuffered(f); long t1 = System.nanoTime();
```

```
            long b = LineCounter.countBuffered(f);
long t2 = System.nanoTime();
            long c = LineCounter.countBufferedStream(f);
long t3 = System.nanoTime();
            System.out.printf("%2d MB  lines=%d  unbuffered=%d ms
byte[8192]=%d ms  BufferedInputStream=%d ms  (check %b)%n",
                    mb, a, (t1 - t0) / 1_000_000, (t2 - t1) /
1_000_000, (t3 - t2) / 1_000_000, a == b && b == c);
            Files.delete(p);
        }
    }
}
```

B1.4 – Measure growth (`GrowthBench.java`)

The program finds the **last** record (worst case, a full scan) 20 times and averages. **Sandbox results:**

Records n	`find` (last record), average
1,000	1.20 ms
10,000	4.51 ms
50,000	11.87 ms
100,000	17.51 ms

Interpretation: `find` scans the file line by line, so the work is O(n): searching is expected to grow roughly in proportion to n. At the large sizes the growth is close to linear (from 50,000 to 100,000 records, ×2 data gave ×1.5 time). At small sizes the measurement is dominated by fixed costs (opening the file, the JIT compiler warming up) and by timer noise, so 1,000 → 10,000 records shows only ×3.8 instead of ×10, so the true linear trend is only visible once the scan dominates. Plotting time against n and fitting a straight line gives the cost per record (here ≈ 0.17 µs). For searches repeated many times, a line scan per query becomes the bottleneck; see the alternative below.

```
import java.io.IOException;
import java.nio.file.*;

public class GrowthBench {
    public static void main(String[] args) throws IOException {
        for (int n : new int[]{1_000, 10_000, 50_000, 100_000}) {
            Path f = Files.createTempFile("g", ".csv");
            try (var w = Files.newBufferedWriter(f)) {
                for (int i = 1; i <= n; i++) w.write("P" + i +
",19231" + i + ",Name" + i + "," + (i % 20) + ",4500.0\n");
            }
            PassStore s = new PassStore(f);
            s.find("P" + n);
// warm-up
            int reps = 20;
```

```
            long t0 = System.nanoTime();
            for (int i = 0; i < reps; i++) s.find("P" + n);
// worst case: last record
            double ms = (System.nanoTime() - t0) / 1e6 / reps;
            System.out.printf("n=%7d  find(last) = %.2f ms%n", n, ms);
            Files.delete(f);
        }
    }
}
```

Alternative concept and complexity

Read the file once into `HashMap<String, String[]>` keyed by pass ID (`IndexedStore.java`, sandbox-tested):

```
import java.io.IOException;
import java.nio.file.*;
import java.util.*;

/** Alternative: read the file once into a HashMap, then every search
is O(1). */
public class IndexedStore {
    private final Map<String, String[]> index = new HashMap<>();
    IndexedStore(Path file) throws IOException {
        try (var r = Files.newBufferedReader(file)) {
            String line;
            while ((line = r.readLine()) != null) {
                String[] f = line.split(",", -1);
                if (f.length == 5) index.put(f[0], f);
            }
        }
    }
    String[] find(String id) { return index.get(id); }

    public static void main(String[] a) throws IOException {
        int n = 100_000, q = 1_000;
        Path f = Files.createTempFile("ix", ".csv");
        try (var w = Files.newBufferedWriter(f)) {
            for (int i = 1; i <= n; i++) w.write("P" + i + ",R" + i +
",Name" + i + "," + (i % 20) + ",4500.0\n");
        }
        PassStore scan = new PassStore(f);
        long t0 = System.nanoTime();
        for (int i = 0; i < 50; i++) scan.find("P" + n);
        double scanMs = (System.nanoTime() - t0) / 1e6 / 50;
        t0 = System.nanoTime();
        IndexedStore ix = new IndexedStore(f);
        double loadMs = (System.nanoTime() - t0) / 1e6;
        t0 = System.nanoTime();
        for (int i = 0; i < q; i++) ix.find("P" + n);
```

```
        double qMs = (System.nanoTime() - t0) / 1e6;
        System.out.printf("scan: %.2f ms per search -> %d searches =
%.0f ms%n", scanMs, q, scanMs * q);
        System.out.printf("map : load %.0f ms once + %d searches =
%.2f ms%n", loadMs, q, qMs);
        Files.delete(f);
    }
}
```

- **Time for q searches on n passes:** scanning each time costs O(q × n); with the map it is O(n) once to build it plus O(1) per lookup: **O(n + q)**.
- **Sandbox measurement (n = 100,000, q = 1,000):** scan ≈ 23.9 ms per search → ≈ 23,900 ms for 1,000 searches; map = 62 ms to load once + 0.77 ms for 1,000 lookups. For this many searches the map approach was hundreds of times faster.
- **Memory:** scanning needs only a line buffer: **O(1)** extra (O(longest line)). The map keeps every record in memory: **O(n)** (100,000 records × five strings per record).
- **When to choose which:** scanning is fine for rare searches or files too large for memory; the map is better when the file is searched many times. Its weaknesses: it can become stale if another program changes the file, and it must be rebuilt (or updated together with the file) after each add.